WAREHOUSE PUTAWAY AND PICKING ENGINE

DESIGN DOCUMENT

System architecture and operational design

A technical overview of the command-line warehouse application, its data model, inventory rules, processing workflow, outputs, and validation approach.

PROJECT DESIGN REPORT

Warehouse Putaway and Picking Engine with FIFO/FEFO and Bin Capacity

1. System architecture

The application is a command-line warehouse processor organized around a command runner and a set of focused Python services:

1. `main.py` parses the command-line arguments and starts the application.
2. `WarehouseApplication` in `src/application.py` loads CSV inputs, constructs shared service instances, dispatches commands, and writes output CSVs.
3. Services in `src/services/` apply inventory, location, putaway, reservation, picking, replenishment, expiry, cycle-count, and audit rules.
4. Domain records in `src/models/entities.py` represent stock lots, movements, reservations, and orders.

The application keeps its working inventory and command state in memory for the duration of one run. Its input and reporting interfaces are CSV files and a pipe-delimited command file.

2. Main modules and responsibilities

Module	Responsibility
`main.py`	Parses the optional command-file path and `--data-dir` and `--output-dir` options; invokes the application and reports rejected commands.
`src/application.py`	Loads input data, composes services, processes commands in order, collects report rows and writes the five output CSVs.
`src/models/entities.py`	Defines `StockLot`, `Movement`, `Reservation`, and `Order` data records.
`src/services/warehouse.py`	Loads and validates zones and bins, and resolves compatible locations.
`src/services/grn.py`	Loads SKU and GRN records, rejects malformed GRN rows, and returns validated lines for receipt.
`src/services/inventory.py`	Maintains stock by bin/SKU/lot; enforces location and capacity constraints; records stock changes.
`src/services/putaway.py`	Selects putaway destinations and routes near-expiry receipts to quarantine.
`src/services/reservation.py`	Reserves eligible stock, orders lots by FIFO/FEFO, and tracks reservation lifecycle.
`src/services/picking.py`	Removes stock for confirmed picks and supports lot selection by FIFO/FEFO.
`src/services/pick_sequence.py`	Orders pick allocations by aisle and alternating bin direction.
`src/services/replenishment.py`	Moves eligible reserve stock to a pick face when its quantity is below threshold.
`src/services/shelf_life.py`	Evaluates minimum remaining shelf life and slow-moving age.
`src/services/expiry.py`	Identifies lots within the seven-day pick block and moves them at end of day.
`src/services/cycle_count.py`	Validates count variances, freezes discrepant bins, and applies adjustments.
`src/services/audit.py`	Exposes movement and stock snapshots and evaluates count variance thresholds.

3. Data model and relationships

- A **zone** has a temperature class and contains bins.
- A **bin** belongs to a zone and has an aisle, bin number, type, unit capacity, and maximum weight. The supported types are `PICK_FACE`, `RESERVE`, and `QUARANTINE`.
- A **SKU** has a unit weight, temperature class, expiry-tracking setting, home zone, and configured pick-face bin.
- A **stock lot** is identified by its bin, SKU, and lot number. It also stores quantity, GRN date, optional manufacturing and expiry dates, and status.
- A **reservation** links an order to a quantity of one SKU/lot in one bin and records creation and expiry timestamps plus confirmation/release state.
- A **movement** records a timestamp, type, SKU, lot, quantity, optional source and destination bins, and a reference such as a GRN or order ID.
- An **order** contains SKU quantities and reservation/pick status.

The `Order` dataclass is available in the domain model. The command runner keeps its final order-report state in an in-memory mapping.

Input CSV relationships are validated where loaded or used: bins must reference existing zones, SKU temperature must match its storage bin, and GRN rows must refer to a known SKU and valid quantities/dates. The inventory key `(bin_id, sku_id, lot_no)` allows the same lot to be tracked independently in different bins.

4. Putaway and bin capacity

Putaway selects temperature-compatible non-quarantine bins for ordinary receipts. Candidate precedence is:

1. A bin already containing the same SKU and lot with matching manufacturing and expiry dates.
2. An empty bin in the SKU's home zone.
3. Another empty temperature-compatible bin.

Candidates within a precedence step are sorted by aisle and bin number. A receipt whose expiry is within 30 days of its receipt date is directed to a compatible quarantine bin instead.

For each destination, available capacity is bounded by both unit capacity and remaining weight divided by SKU unit weight. A bin may hold at most two distinct SKUs. A pallet may be split across bins, but the service skips a candidate if using its capacity would leave a remainder of fewer than 10 units; a remainder of exactly 10 is permitted. If the full receipt cannot be placed under these rules, the command is rejected; the command runner rolls back the partial inventory, movement, and putaway-row effects for that receipt.

5. FIFO and FEFO picking

The reservation service builds the order allocation before confirmation, so competing orders cannot reserve the same available quantity:

- **FIFO:** Non-expiry-tracked lots are sorted by GRN date, then bin ID. Lots marked `SLOW` sort after all non-`SLOW` lots; within each group the same GRN-date/bin ordering applies.
- **FEFO:** Expiry-tracked lots are sorted by earliest expiry date, then bin ID. An order's shelf-life check is inclusive: expiry date minus order date must be at least the requested minimum.

- The default minimum remaining shelf life is 60 days; `ORDER` may specify a non-negative alternative. A missing expiry date does not qualify an expiry-tracked lot for reservation.
- Independently of the order minimum, the inventory pickability filter excludes expiry-tracked lots with seven or fewer days remaining.

Non-expiry-tracked stock becomes `SLOW` only when its age from GRN date is strictly greater than 90 days. The runner performs an initial slow-stock refresh for the opening-stock date (2028-01-17 in this application) and refreshes it again at each `EOD`. Expiry-tracked stock is not marked slow by this rule.

After an order is confirmed, the runner validates that each reservation is still available and removes the reserved quantity. Completed reservations are then released from the reservation ledger so already-consumed stock does not remain locked.

`PickSequenceService` orders allocations by ascending aisle, starting with ascending (`UP`) bin numbers in the lowest aisle represented in the order. The direction for each allocation is determined by the parity of its aisle's offset from that lowest aisle: even offsets use `UP`, odd offsets use descending (`DOWN`) bin numbers. Thus direction alternates by physical aisle, including when an intervening aisle has no pick. The output includes a sequence number, direction, and count of distinct aisles in that order's pick list.

6. Reservations and replenishment

Reservations expire 48 hours after creation. A reservation is expired at the exact expiry timestamp (`expires_at <= current time`); confirmation requires that it has not expired. At reservation time, quantities held by other unconfirmed reservations that have not expired and confirmed reservations that have not been released are excluded from available stock. Unconfirmed expired reservations are released during reservation housekeeping. Confirmed reservations remain protected even beyond 48 hours until their stock is picked or the order is released; after successful picks the runner marks them as picked and releases their locks.

After a successful pick, the runner checks the configured pick face for each SKU in the order. Replenishment is triggered only when pick-face on-hand quantity is strictly below 20% of unit capacity. The replenishment service targets 80% of unit capacity, limited by remaining unit capacity, weight capacity, and eligible reserve stock; it may end below 80% if any of those limits prevent a full refill. For FIFO replenishment, lots sort by slow-stock status, GRN date, bin ID, then lot number. For FEFO replenishment, lots sort by expiry date, bin ID, then lot number. It selects only compatible `RESERVE` bins; quarantine and frozen-bin stock is not used. See the limitations section for the expiry-age filter applied to replenishment.

Each successful replenishment transfer is retained in the movement ledger and exported as `REPLENISH`.

7. Expiry and quarantine

For expiry-tracked stock, lots with seven or fewer days remaining are unpickable. The end-of-day (`EOD`) command processes the supplied date at 23:59:59 and physically transfers eligible due lots to a temperature-compatible quarantine bin. The resulting movement is reported as `QUARANTINE`; the stock record in the destination has `QUARANTINE` status.

Receipts expiring within 30 days of receipt are routed directly to quarantine during putaway and therefore do not enter ordinary pickable stock.

8. Cycle counting and audit trail

Cycle-count input rows identify a count batch, bin, SKU, lot number, and counted quantity. The implementation interprets `counted_qty` as the total quantity for that SKU in the bin rather than a quantity for the named lot. `lot_no` is retained to assign a positive variance. A shortfall is subtracted from the oldest eligible lot first.

The absolute variance is compared with the smaller of 2% of system quantity and five units: `min(0.02 * system_qty, 5)`. A variance freezes the bin only when its absolute value is strictly greater than that threshold; equality does not freeze. The system quantity includes `AVAILABLE` and `SLOW` stock, and shortage adjustments use those eligible lots in ascending GRN-date and lot number order. A freezing variance queues its adjustment until `ADJUST` is issued; a variance that does not require a freeze is adjusted immediately. Adjustments and all other stock changes are recorded in the movement ledger with their simulated timestamps.

`PRINT_AUDIT` captures the movement rows for the requested SKU at the point in the command stream when the request occurs. The CLI displays those captured rows. `PRINT_SOH` captures positive-quantity stock records at its command timestamp; the stock report is sorted by SKU, lot, and bin.

9. Command-processing workflow

The command file uses `COMMAND_NAME|arg1|arg2|...`. Blank lines and lines beginning with `#` are ignored. Commands are handled sequentially:

1. Load zones, bins, SKUs, GRN lines, opening stock, and cycle-count rows.
2. Initialize the starting inventory without creating movement records for opening stock, then mark the applicable older non-expiry stock as slow.
3. Parse and dispatch each command, using its provided timestamp or date.
4. Apply the corresponding service operations and record successful stock movements and report rows.
5. Record command-level `KeyError`/`ValueError` rejections, continue with later commands, and write all five CSV outputs at the end of the run.

The runner supports the following eight commands:

Command	Arguments after command name	Effect
`RECEIVE`	`grn_id|ts`	Validates and puts away the GRN lines. Duplicate receipts are rejected.
`ORDER`	`order_id|ts|customer_id|allow_short|min_shelf_life_days|lines`	Reserves available eligible stock; lines use `SKU:qty` entries separated by commas. An empty minimum shelf-life argument uses 60 days. With `allow_short=false`, insufficient eligible stock holds the order; with `true`, available quantities may be reserved and the remainder is reported as short.
`CONFIRM`	`order_id|ts`	Confirms active reservations, picks the reserved stock, then checks replenishment.
`EOD`	`date`	Refreshes slow-stock status, moves eligible expiry-blocked stock to quarantine, and releases expired unconfirmed reservations.
`CYCLE_COUNT`	`count_batch|ts`	Applies every count row in the named batch.
`ADJUST`	`bin_id|ts`	Applies pending variance adjustments for a frozen bin and unfreezes it.
`PRINT_SOH`	`ts`	Captures positive stock-on-hand rows.
`PRINT_AUDIT`	`sku_id`	Captures the movement rows for that SKU at that point in the command stream.

Argument order and detailed input contracts are documented in `../data/README.md`.

10. Output files

The runner writes these five CSV files to the configured output directory:

File	Columns, in order	Rows and ordering
`putaway.csv`	`grn_id`, `line_no`, `sku_id`, `lot_no`, `bin_id`, `qty`, `precedence_step`, `rule_ids`	One row per bin used for a pallet or pallet portion, in receipt/putaway processing order.
`pick_lists.csv`	`order_id`, `seq`, `bin_id`, `sku_id`, `lot_no`, `qty`, `aisle`, `walk_direction`, `total_aisles_walked`	One row per pick allocation, in the generated pick sequence.
`order_status.csv`	`order_id`, `status`, `reserved_at`, `released_at`, `lines_short`, `rule_ids`	One final row per order, sorted by order ID.
`movements.csv`	`ts`, `movement_type`, `sku_id`, `lot_no`, `from_bin`, `to_bin`, `qty`, `reference`	Every recorded stock movement, in processing order; empty source/destination represents movement outside the warehouse.
`stock_on_hand.csv`	`sku_id`, `lot_no`, `bin_id`, `qty`, `status`	Positive-quantity rows captured by `PRINT_SOH`, sorted by SKU, lot, then bin.

11. Validation and testing

The pytest suite includes focused service tests for warehouse validation, capacity, GRN validation, FIFO/FEFO, shelf life, reservation expiry, serpentine sequencing, replenishment, expiry handling, cycle-count thresholds, and audit behavior. Application integration tests exercise duplicate receipt handling, frozen-bin selection, the EC-06 replenishment sequence, command-time audit capture, output headers and ordering, and the 172-command sample stream.

Run the complete test suite from the repository root with:

```
pytest -q
```

The sample has three intentional rejected commands: a duplicate GRN receipt, a receipt that cannot be placed within the capacity/split rules, and a GRN with no valid lines. The integration test checks these expected rejections and confirms that the remaining sample commands are processed.

12. Assumptions and limitations

- Cycle counts are interpreted at SKU/bin aggregate level because the variance rule compares system stock for a SKU in a bin. The supplied `lot_no` is used to attribute positive adjustments; shortages use oldest-lot-first.
- Opening stock initializes balances but is not included as movements in the audit output.
- Replenishment orders expiry-tracked reserve lots by FEFO, but the current replenishment service does not filter those source lots by remaining shelf life or the seven-day pick block. Reservation-time pickability still blocks such stock from customer picks, and `EOD` quarantines eligible blocked stock.
- The application has no database or persistent reservation store; state is held in memory for one command-file run.
- Each `PRINT_SOH` call captures a snapshot, but the CSV schema has no snapshot timestamp; the runner writes the captured snapshots as rows without a snapshot identifier. The supplied sample contains one `PRINT_SOH` command.
- Command-level rejections are printed and processing continues. A run can therefore write outputs after rejected commands; review stderr to see those rejections. The CLI currently exits successfully even when commands were rejected.
- The sample-data documentation states that reference output files are not supplied. Tests validate implemented rules and output contracts, but there is no provided golden CSV set for field-by-field comparison against an external reference.
- The root-level prototype modules remain in the repository. The active CLI uses the `src/` package services, not those prototype modules.